

Projeto de Simulação Dinâmica HIL (Hardware-in-the-Loop)

Interface Serial PC-G29, ESP32 Clássico e ECU Speeduino

Instituição:

Instituto Federal de Minas Gerais – Campus Betim

Autor:

William Las-Cazas Queiroz (0104566)

Capitão e Gerente Técnico - Equipe Elétrica

Aplicação:

Validação Dinâmica de Calibração HIL em Malha Fechada

22 de outubro de 2026

Sumário

1 Introdução ao Simulador HIL **3**

2 Lista de Materiais (BOM de Bancada) **3**

3 Arquitetura Lógica de Integração HIL **4**

4 Mapeamento do Chicote (ESP32 ↔ Speeduino) **4**

4.1 Lógica de Tradução de Sinais (Geração) 4

4.2 Lógica de Feedback e Realimentação (Leitura da ECU) 4

5 Diagrama Visual das Conexões Físicas **6**

5.1 Envio de Sinais: Conector Preto (B) 6

5.2 Retorno de Sinais (Feedback): Conector Cinza (G) 6

6 Arquitetura Base do Firmware (ESP32 via Serial) **6**

1 Introdução ao Simulador HIL

Este documento descreve a elevação dos testes de bancada da Speeduino 48-Way para o nível de simulação dinâmica orientada ao condutor (Hardware-in-the-Loop). A arquitetura atualizada delega a leitura do volante Logitech G29 a um Computador (PC), que atua como *host* USB e transmite os comandos do piloto (acelerador, embreagem e marcha) via comunicação Serial (UART) para um microcontrolador.

O cérebro da planta física é um **ESP32 Clássico (30 pinos)**. Ele recebe a telemetria do PC, calcula o modelo matemático da dinâmica do motor e converte essas variáveis em sinais elétricos que excitam a ECU em tempo real. A malha de simulação é fechada lendo as respostas de injeção e ignição geradas pela Speeduino através de interrupções.

2 Lista de Materiais (BOM de Bancada)

- **1x Computador (PC):** Executa o script (ex: Python) que lê o USB do G29 e envia a string de dados via porta COM.
- **1x Placa Devkit ESP32 (30 pinos):** Executa a física do motor, converte dados da Serial em tensão/pulsos e lê a ECU.
- **1x Módulo DAC I2S PCM5102A:** Dedicado exclusivamente à síntese matemática da roda fônica 22-2 (Crank).
- **Capacitores e Resistores (Filtro RC):** Como o ESP32 possui DACs internos de 8-bits, são necessários capacitores na saída analógica para suavizar os degraus de tensão antes de entrarem na ECU.
- **Módulos de Isolamento (Optoacopladores):** Componentes rápidos (ex: 6N137 ou PC817 com pull-up adequado) para condicionar os sinais de 5V/12V de injeção e ignição para 3.3V.

Nota: A síntese de áudio do motor e o hardware USB Host foram removidos para simplificar e garantir a estabilidade do ciclo principal da física no ESP32.

3 Arquitetura Lógica de Integração HIL

A dinâmica do simulador foi modelada sob a ótica de sistemas de controle em malha fechada, com o PC atuando como ponte de aquisição de dados.

O sistema divide-se em três blocos principais:

- **Interface Humana (PC + G29):** O PC lê os pedais e botões, empacota os dados e os envia a 115200 *baud* para o ESP32.
- **Planta Física (ESP32):** Lê o pacote Serial, resolve numericamente o balanço entre o torque indicado e o torque resistivo veicular. A integral do torque líquido define a rotação virtual (ω_m), convertida em sinais físicos utilizando os conversores DAC de 8-bits nativos (pinos 25 e 26), PWM e I2S.
- **Controlador (Speeduino):** Lê os sensores virtuais. A realimentação da malha ocorre quando as saídas da ECU são interceptadas pelos pinos de interrupção (ISR) do ESP32 e reinjetadas no modelo de combustão.

4 Mapeamento do Chicote (ESP32 $\leftrightarrow$ Speeduino)

4.1 Lógica de Tradução de Sinais (Geração)

1. **Acelerador (TPS):** A porcentagem do pedal (recebida via Serial) alimenta o **DAC1 (Pino 25)**. A tensão passa por um filtro RC (resistor-capacitor) para suavizar a resolução de 8-bits e é injetada na porta **B_C7 (TPS_Sensor)**.
2. **Sensor MAP:** A pressão virtual simulada alimenta o **DAC2 (Pino 26)**, também com filtro RC na saída, sendo injetada em **B_C8 (MAP-EXT)**.
3. **Giro do Motor (Crank 22-2):** O I2S via DMA gera a roda fônica no PCM5102A para a porta **B_B2 (CRANK-IN)**.
4. **Velocidade (VSS) e Embreagem:** PWM injetado em **B_A7 (VSS)**, e um GPIO (ex: Pino 15) acionando **B_B7 (Clutch_in)**.

4.2 Lógica de Feedback e Realimentação (Leitura da ECU)

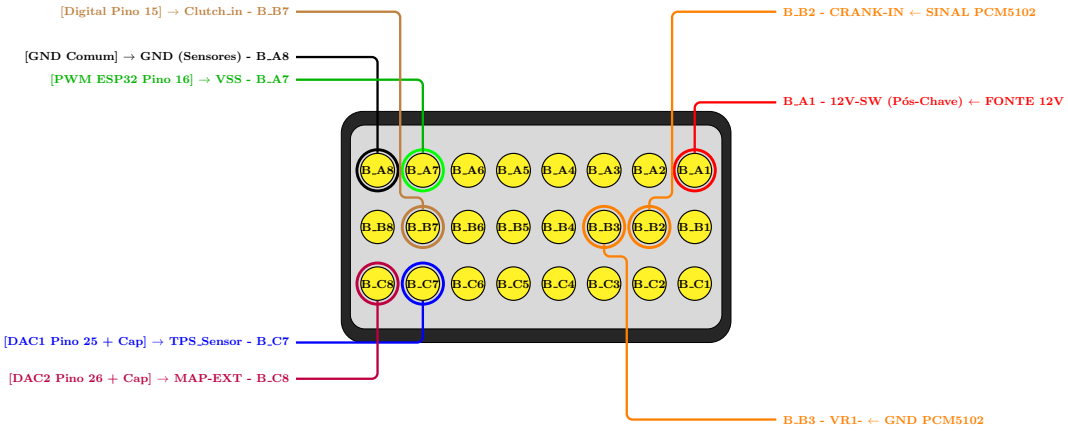
O ESP32 intercepta os eventos críticos através de Interrupções de Hardware (ISR):

1. **Bico Injetor 1:** O pino **G_A1 (INJ-1-OUT)** (Low-side). O ESP32 mede a largura do pulso em microssegundos.

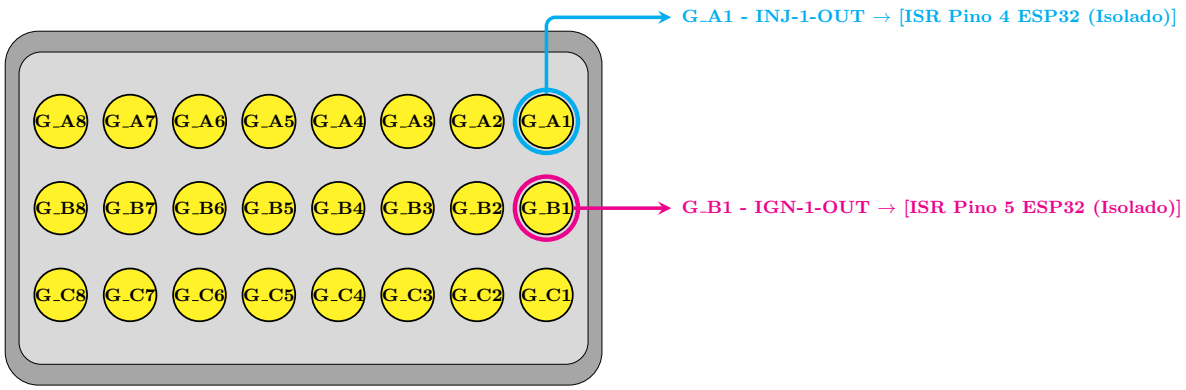
2. **Bobina 1 (Avanço):** O pino **G_B1 (IGN-1-OUT)** (Smart Coil 5V). O ESP32 detecta o momento do disparo da centelha.

5 Diagrama Visual das Conexões Físicas

5.1 Envio de Sinais: Conector Preto (B)



5.2 Retorno de Sinais (Feedback): Conector Cinza (G)



6 Arquitetura Base do Firmware (ESP32 via Serial)

O código demonstra a recepção dos dados (acelerador, embreagem, marcha) vindos do PC pela porta Serial e a utilização dos DACs internos de 8-bits com suporte às interrupções de velocidade crítica (Injeção/Ignição).

```
#include <Arduino.h>

// Pinos de Saida (ESP32 -> Speeduino)
#define DAC_TPS_PIN 25 // Necessario Filtro RC (Capacitor para terra)
#define DAC_MAP_PIN 26 // Necessario Filtro RC (Capacitor para terra)
#define CLUTCH_PIN 15
```

```
#define VSS_PIN 16
#define VSS_PWM_CHANNEL 0

// Pinos de Entrada (Speeduino -> ESP32) - Malha Fechada
#define FEEDBACK_INJ_PIN 4
#define FEEDBACK_IGN_PIN 5

// Variaveis de Estado HIL
uint16_t current_rpm = 1000;
uint8_t current_gear = 0;
float gear_ratios[7] = {0.0, 2.66, 1.88, 1.45, 1.20, 1.03, 0.90};

volatile uint32_t tempo_injecao_us = 0;
volatile uint32_t inicio_pulso_inj = 0;
volatile uint32_t momento_ignicao_us = 0;

// ========================================
// ROTINAS DE INTERRUPCAO (ISR)
// ========================================
void IRAM_ATTR isr_leitura_injetor() {
    if (digitalRead(FEEDBACK_INJ_PIN) == LOW) {
        inicio_pulso_inj = micros();
    } else {
        tempo_injecao_us = micros() - inicio_pulso_inj;
    }
}

void IRAM_ATTR isr_leitura_ignicao() {
    momento_ignicao_us = micros();
}

// Processa o pacote recebido do PC: Formato "throttle,clutch,gear"
void processarDadosDoPC() {
    if (Serial.available() > 0) {
        String data = Serial.readStringUntil('\n');

        int firstComma = data.indexOf(',');
        int secondComma = data.indexOf(',', firstComma + 1);
```

```
        if (firstComma > 0 && secondComma > 0) {
            uint8_t raw_throttle = data.substring(0, firstComma).toInt
    ();
            uint8_t raw_clutch = data.substring(firstComma + 1,
    secondComma).toInt();
            current_gear = data.substring(secondComma + 1).toInt();

            // Embreagem (On/Off)
            digitalWrite(CLUTCH_PIN, (raw_clutch > 127) ? HIGH : LOW);

            // Escreve a tensao analogica nos DACs internos (8-bits: 0
    a 255)
            dacWrite(DAC_TPS_PIN, raw_throttle);

            // Simulacao do MAP com base no acelerador (Pode ser
    calculado via motor)
            uint8_t map_val = map(raw_throttle, 0, 255, 75, 250); //
    Mapeado para 0-255
            dacWrite(DAC_MAP_PIN, map_val);

            // Atualizacao simples do RPM baseada no HIL
            current_rpm = map(raw_throttle, 0, 255, 1000, 10500);

            // Calculo da velocidade (VSS)
            if (current_gear > 0 && raw_clutch < 127) {
                uint16_t vss_hz = (current_rpm / gear_ratios[
    current_gear]) / 60;
                ledcWriteTone(VSS_PWM_CHANNEL, vss_hz);
            } else {
                ledcWriteTone(VSS_PWM_CHANNEL, 0);
            }
        }
    }
}

void setup() {
    Serial.begin(115200);

    pinMode(CLUTCH_PIN, OUTPUT);
```

```
    ledcSetup(VSS_PWM_CHANNEL, 0, 8);
    ledcAttachPin(VSS_PIN, VSS_PWM_CHANNEL);

    pinMode(FEEDBACK_INJ_PIN, INPUT_PULLUP);
    pinMode(FEEDBACK_IGN_PIN, INPUT_PULLDOWN);

    attachInterrupt(digitalPinToInterrupt(FEEDBACK_INJ_PIN),
   isr_leitura_injetor, CHANGE);
    attachInterrupt(digitalPinToInterrupt(FEEDBACK_IGN_PIN),
   isr_leitura_ignicao, RISING);

    // setupI2SCrank(); // Iniciar geracao da roda fonica 22-2 no
   PCM5102A
}

void loop() {
    processarDadosDoPC();
    // O PC tambem lera pacotes do ESP32 (ex: RPM atual) pela Serial
   para vibrar o G29
}
```

Listing 1: Integração Serial e Malha Fechada no ESP32